Design and Analysis of

Algorithms ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database

containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis

- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**

- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of

the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency

- Its experimentally observed performance
- Any preprocessing cost involved

- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day.**

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected

- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution. If

you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis

- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

TASK 1 – Design the Algorithm

1.1 Introduction

The university maintains a large student database containing registration numbers. The database size may increase from 1,000 to 1,000,000 or more records. Therefore, an efficient searching algorithm is required to locate a student's record quickly. Three different approaches are proposed: Linear Search, Recursive Binary Search, and Hash Table Search.

1.2 Approach 1: Linear Search

Basic Idea: Linear Search checks each registration number one by one until the required number is found or the list ends. It works on both sorted and unsorted data. **Design Strategy:** Brute-force search. **Preprocessing Required:** No preprocessing or sorting is required. **Pseudocode:**

Code

```
LinearSearch(A, n, key)

1. For i = 0 to n - 1

2.     If A[i] == key

3.         Return i

4. Return -1
```

Explanation: The algorithm begins with the first registration number and compares it with the required key. If both are equal, it returns the position. Otherwise, it continues to the next element. If the key is not present after checking all elements, it returns -1. **Justification for Selection:** Linear Search is selected because it is simple to implement and works directly on unsorted student records. It provides a baseline for comparing the performance of more efficient searching methods. However, the number of comparisons can increase significantly when the database becomes large.

1.3 Approach 2: Recursive Binary Search

Basic Idea: Binary Search repeatedly divides a sorted list into two halves. It compares the required registration number with the middle element and continues searching only in the relevant half. **Design Strategy:** Divide and conquer. **Preprocessing Required:** The registration numbers must be sorted before searching. The sorting cost must be included in the total efficiency analysis if sorting is required. **Pseudocode:**

Code

```
BinarySearch(A, low, high, key)

1. If low > high

2.     Return -1
```

3. mid = (low + high) // 2

4. If A[mid] == key

5. Return mid

6. Else if key < A[mid]

7. Return BinarySearch(A, low, mid - 1, key)

8. Else

9. Return BinarySearch(A, mid + 1, high, key)

Explanation: The algorithm calculates the middle position and compares the middle registration number with the required key. If they match, the search ends. If the key is smaller, the algorithm searches the left half; otherwise, it searches the right half. The process continues recursively until the key is found or the search range becomes empty. **Justification for Selection:** Recursive Binary Search is selected because it reduces the search space by half at each step. It is suitable for large, sorted databases where records are searched frequently. Its main limitation is the requirement for sorted data and the associated preprocessing cost.

1.4 Approach 3: Hash Table Search

Basic Idea: A hash table uses a hash function to calculate the storage location of each registration number. During searching, the same function identifies the location where the required record should be stored. **Design Strategy:** Hashing. **Preprocessing Required:** A hash table must be constructed by inserting the registration numbers. The construction cost and memory requirements must be considered. **Pseudocode:**

Code

HashSearch(H, key)

1. index = HashFunction(key)

2. If H[index] contains key

3. Return index

4. Search the appropriate collision chain or probe sequence

5. If key is found

6. Return its position

7. Return -1

Explanation: The hash function maps the registration number to an index in the hash table. If the key is stored at that location, the record is returned immediately. If multiple keys map to the same location, a collision-resolution method such as chaining or open addressing is used. **Justification for Selection:** Hash Table Search is selected because it can provide very fast average search performance without requiring the registration numbers to be sorted. It is

useful when many searches are performed on a database that changes relatively infrequently. Its performance depends on the hash function, collision handling, load factor, and memory availability.

1.5 Comparison of the Proposed Approaches

Linear Search examines records sequentially and does not require preprocessing. Recursive Binary Search repeatedly halves the search range but requires sorted data. Hash Table Search uses calculated storage locations and requires construction of the hash table. These approaches represent three distinct strategies: brute force, divide and conquer, and hashing. They will be implemented and compared using increasing dataset sizes to determine their suitability for the university database.

1.6 Conclusion

The three proposed algorithms provide different ways to search student registration numbers. Linear Search is simple and works with unsorted records, Recursive Binary Search is efficient for sorted records, and Hash Table Search is suitable for frequent lookups. Their mathematical complexities, preprocessing costs, and experimental performance will be evaluated in the following tasks to identify the most appropriate solution.

TASK 2 – Identify the Basic Operation

2.1 Introduction

The basic operation is the most frequently executed operation in an algorithm. It is used to analyse the efficiency of an algorithm by counting how many times the operation is performed as the input size increases. In the university student record retrieval system, the input size $n$represents the total number of student records, and the search key represents the registration number that must be located. The three proposed algorithms use different strategies, so their basic operations and execution counts are also different. Identifying these operations helps compare the algorithms mathematically and understand their performance for databases containing 1,000, 10,000, 100,000, and 1,000,000 or more records.

2.2 Linear Search

Basic Operation: Comparison between the current registration number and the required search key.

Where It Occurs: The basic operation occurs inside the loop at the condition A[i] == key. The algorithm checks each registration number sequentially, starting from the first element.

Why It Is Appropriate for Efficiency Analysis: Each comparison determines whether the required registration number has been found. If the values are not equal, the algorithm moves to the next record. Therefore, the total number of comparisons directly depends on the position of the key and the size of the database. Counting these comparisons provides a clear way to measure the algorithm's execution.

Effect of Input Size (n): In the best case, the required registration number is present at the first position, so only one comparison is needed. In the average case, the key may be located somewhere in the middle of the list, requiring approximately $(n + 1)/2$comparisons when the key is equally likely to be at any position. In the worst case, the key is at the last position or is not present, so the algorithm performs $n$comparisons.

The number of comparisons can be represented as follows:

Best case: $C(n) = 1$

Average case: $C(n) = (n + 1)/2$

Worst case: $C(n) = n$

For a database containing 1,000 records, the worst-case number of comparisons is 1,000. For 10,000 records, it is 10,000. For 100,000 records, it is 100,000. For 1,000,000 records, it can reach 1,000,000 comparisons.

This shows that the number of comparisons increases directly with the number of records. Linear Search is simple and does not require preprocessing, but its performance becomes less suitable when the database is very large and searches are performed frequently.

2.3 Recursive Binary Search

Basic Operation: Comparison between the middle registration number and the required search key.

Where It Occurs: The basic operation occurs when the algorithm compares A[mid] with the search key. If they are not equal, another comparison determines whether the key is smaller or larger than the middle element. The algorithm then recursively searches only the relevant half.

Why It Is Appropriate for Efficiency Analysis: Binary Search eliminates approximately half of the remaining search elements after each step. The middle-element comparison determines whether the required record has been found and which subproblem must be processed next. Therefore, counting these comparisons explains why Binary Search performs fewer operations than Linear Search for large, sorted datasets.

Effect of Input Size (n): The number of comparisons increases logarithmically because the search range is repeatedly divided by two. After one step, approximately $n/2$elements remain. After two steps, approximately $n/4$remain. After three steps, approximately $n/8$remain. After $k$steps, approximately $n/2^k$elements remain.

The search ends when the remaining range is empty or the key is found. The approximate maximum number of halving steps is determined by $\log_2 n$.

For example:

- For 1,000 records, $\log_2(1000) \approx 9.97$, so approximately 10 halving steps are required.

- For 10,000 records, $\log_2(10000) \approx 13.29$, so approximately 14 halving steps may be required.
- For 100,000 records, $\log_2(100000) \approx 16.61$, so approximately 17 halving steps may be required.
- For 1,000,000 records, $\log_2(1000000) \approx 19.93$, so approximately 20 halving steps may be required.

The number of comparisons therefore grows much more slowly than the number of records. However, Binary Search requires the records to be sorted by registration number. If sorting is necessary, its preprocessing cost must be included when analysing the total cost of the system. For a database that is searched frequently but updated only occasionally, this preprocessing may be worthwhile.

2.4 Hash Table Search

Basic Operation: Hash-function computation, followed by key comparison and any additional collision-resolution operations.

Where It Occurs: The hash function calculates an index for the required registration number. The algorithm then checks the corresponding table location. If the key is not located there because of a collision, the algorithm checks additional positions or follows the relevant collision chain.

Why It Is Appropriate for Efficiency Analysis: Hash Table Search does not normally examine every record sequentially. Instead, it calculates a likely storage location directly. The number of additional operations depends on the quality of the hash function, the number of collisions, the load factor, and the collision-resolution method. Therefore, the number of key comparisons and collision-handling steps is useful for measuring its efficiency.

Effect of Input Size (n): When a suitable hash function distributes registration numbers evenly and the table maintains a controlled load factor, searching typically requires a small, approximately constant number of operations on average. Thus, average search time is expected to be $O(1)$.

However, this performance is not guaranteed for every input. If many registration numbers map to the same location, the algorithm may need to examine several entries. In the worst case, a large number of collisions can cause the search time to reach $O(n)$.

For example, with 1,000 records and good hash distribution, a search may require only a small number of key comparisons. With 1,000,000 records, the average search may still require only a small number of operations if the table is appropriately sized and the hash function performs well. Actual performance must be measured using the implemented hash table and its collision-resolution method.

The construction of the hash table must also be considered. Inserting all $n$records generally takes expected $O(n)$total time with suitable hashing, while poor collision behaviour can

increase the construction cost. The table also requires additional memory to store registration numbers and their corresponding record locations.

2.5 Comparison of Basic Operations

Algorithm	Basic operation	Typical average search behaviour	Worst-case behaviour
Linear Search	Compare each record with the key	Approximately $(n + 1)/$ 2comparisons for a successful search with equally likely positions	$n$comparisons
Recursive Binary Search	Compare the middle element with the key	Logarithmic number of comparisons	Logarithmic number of comparisons
Hash Table Search	Compute hash index and compare keys	Expected constant-time search with good hashing	Linear time if collisions are severe

The comparison shows that the algorithms differ in how they locate the required registration number. Linear Search depends on the position of the record in the list. Binary Search reduces the search range at every recursive step. Hash Table Search uses a calculated index and depends on effective distribution and collision handling.

2.6 Relationship Between Basic Operations and Database Growth

As the university database increases from 1,000 to 1,000,000 records, the number of basic operations becomes an important factor in deciding which algorithm to use. Linear Search may need to inspect a large portion of the database for each request. Binary Search requires far fewer comparisons, provided the records are sorted. Hash Table Search can provide fast average lookups, provided that the table is constructed and maintained appropriately.

The database also receives new records, so preprocessing and maintenance must be considered alongside search time. Linear Search does not require sorting, Binary Search requires sorted data, and Hash Table Search requires a constructed table. The best choice therefore depends not only on search complexity but also on how often searches occur, how frequently records are added, and how much memory is available.

2.7 Conclusion

The basic operation provides a foundation for analysing the three proposed algorithms. Linear Search uses sequential key comparisons, Recursive Binary Search uses middle-element comparisons to reduce the search space, and Hash Table Search uses hashing and key comparisons to locate records. Their operation counts grow differently as the database

becomes larger. These differences will be used in the next task to derive the mathematical time-complexity functions, including the cost of preprocessing.

TASK 3 – Derive the Time-Complexity Function Mathematically

3.1 Introduction

Time complexity describes how the execution time of an algorithm grows as the input size increases. In this assignment, the input size $n$represents the number of student records in the university database. The purpose of mathematical analysis is to determine the number of basic operations performed by each algorithm and express the running time as a function of n. The three algorithms considered are Linear Search, Recursive Binary Search, and Hash Table Search. The analysis also includes preprocessing costs, such as sorting the records or constructing the hash table, because these operations affect the total cost of the system.

3.2 Time-Complexity Function of Linear Search

Input Size: Let $n$be the total number of student records, and let the search key be the registration number to be located.

Basic Operation: Comparing the current registration number with the search key.

Mathematical Derivation: Linear Search starts at the first element and compares each record with the required key. If the key is found at position i, using zero-based indexing, the algorithm performs $i + 1$comparisons. If the key is absent, it performs $n$comparisons.

In the best case, the key is found at the first position.

$$C_{\text{best}}(n) = 1$$

In the worst case, the key is at the last position or is absent.

$$C_{\text{worst}}(n) = n$$

If a successful search is equally likely to find the key at any of the $n$positions, the average number of comparisons is:

$$C_{\text{avg}}(n) = \frac{1}{n}\sum_{i=1}^{n} i$$

Using the formula $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$:

$$C_{\text{avg}}(n) = \frac{1}{n} \times \frac{n(n+1)}{2}$$

$$C_{\text{avg}}(n) = \frac{n+1}{2}$$

Therefore, the average number of comparisons is approximately $n/2$. If unsuccessful searches are also considered, their probability must be specified to calculate a combined average.

Running-Time Function: If each comparison requires constant time c, the worst-case running time can be expressed as:

$$T(n) = cn + d$$

Here, $c$represents the time per comparison, and $d$represents fixed overhead. The linear term dominates as $n$increases.

Preprocessing Cost: Linear Search does not require sorting or restructuring the records. Therefore, the preprocessing cost is $P(n) = 0$, excluding any cost required to load the database into memory.

Overall Cost: The total cost is the preprocessing cost plus the search cost.

$$T_{\text{total}}(n) = P(n) + T(n)$$

For Linear Search, the worst-case cost is:

$$T_{\text{total}}(n) = 0 + cn + d$$

Therefore, the worst-case time complexity is $O(n)$, and the worst-case growth is $\Theta(n)$for the standard implementation.

Interpretation: Linear Search is suitable for small or unsorted datasets because it requires no sorting. However, the number of comparisons increases directly with the number of records. For a database of 1,000,000 records, an unsuccessful search may require 1,000,000 comparisons. This makes Linear Search less suitable when the university must handle a large number of search requests.

3.3 Time-Complexity Function of Recursive Binary Search

Input Size: Let $n$be the number of sorted student records being searched.

Basic Operation: Comparing the middle registration number with the required key.

Mathematical Derivation: Binary Search calculates the middle position and compares its value with the search key. If the key is not found, only one half of the current search range is processed in the next recursive call. Thus, the problem size is reduced from $n$to approximately $n/2$.

After one step, the remaining search size is approximately $n/2$.

After two steps, the remaining search size is approximately $n/4$.

After three steps, the remaining search size is approximately $n/8$.

After $k$steps, the remaining size is approximately $n/2^k$.

The search terminates when the remaining range contains the key or becomes empty. At the termination point:

$$n/2^k \leq 1$$

Multiplying both sides by 2^k:

$$n \leq 2^k$$

Taking logarithms to base 2:

$$\log_2 n \leq k$$

Therefore:

$$k \geq \log_2 n$$

This demonstrates that the number of recursive steps increases logarithmically with the input size.

Running-Time Function: Each recursive call performs a constant amount of work outside the recursive call, including calculating the middle index and performing key comparisons. The time-complexity recurrence is:

$T(n) = T(n/2) + c$, for $n > 1$

The base case is:

$$T(1) = d$$

Here, $c$represents the constant work performed in each recursive call, and $d$represents the base-case cost. Solving this recurrence gives a logarithmic running-time function.

$$T(n) = c\log_2 n + d$$

Therefore, the search time complexity is $O(\log n)$, and its worst-case growth is $\Theta(\log n)$.

Preprocessing Cost: Binary Search requires the registration numbers to be sorted. If the original records are unsorted, a comparison-based sorting algorithm such as Merge Sort can be used. Its typical time complexity is $O(n \log n)$.

The preprocessing cost is therefore:

$$P(n) = O(n \log n)$$

If the records are already sorted, this additional sorting cost is not required for each search.

Overall Cost: If sorting is required before a single search, the total cost is:

$$T_{\text{total}}(n) = T_{\text{sort}}(n) + T_{\text{search}}(n)$$

$$T_{\text{total}}(n) = O(n \log n) + O(\log n)$$

$$T_{\text{total}}(n) = O(n \log n)$$

If the database is sorted once and reused for many searches, the sorting cost can be shared across those searches. Each subsequent search then takes $O(\log n)$time, provided the data remains sorted.

Interpretation: Recursive Binary Search is highly efficient for searching large, sorted datasets. A database containing 1,000,000 records requires approximately 20 halving steps in the worst case. However, its suitability depends on the cost of maintaining sorted records when new student registrations are added.

3.4 Time-Complexity Function of Hash Table Search

Input Size: Let $n$be the number of student records stored in the hash table.

Basic Operation: Computing the hash index and comparing the required key with the stored key. Additional operations may be needed to handle collisions.

Mathematical Derivation: Hash Table Search applies a hash function to the registration number to calculate a storage index. With a suitable hash function, the required record can usually be located by examining a small number of entries.

Let $c_h$represent the constant time needed to compute the hash index and perform the initial lookup. In the ideal case, no collision occurs, and the key is found at the calculated location.

$$T_{\text{best}}(n) = c_h$$

In the average case, assuming a suitable hash function and a controlled load factor, the expected number of comparisons remains approximately constant.

$$T_{\text{avg}}(n) \approx c_h + c_c$$

Here, $c_c$represents the expected additional cost of checking entries affected by collisions. Under suitable hashing conditions, this cost remains bounded on average.

Therefore, the expected average search time is $O(1)$.

However, if many registration numbers map to the same location, the algorithm may need to examine a long collision chain or probe many positions. In the worst case:

$$T_{\text{worst}}(n) = an + b$$

Here, $a$represents the cost per additional entry examined, and $b$represents fixed overhead. The worst-case time complexity is therefore $O(n)$.

Preprocessing Cost: The hash table must be constructed by inserting all student registration numbers. With a suitable hash function and efficient collision handling, the expected total construction time is $O(n)$.

$P(n) = O(n)$(expected)

If the table is poorly designed or experiences severe collisions, construction can take longer. The memory required for the table must also be considered.

Overall Cost: When the table is constructed once and reused for searching, the total cost over $q$searches can be represented as:

$$T_{\text{total}}(n, q) = T_{\text{build}}(n) + qT_{\text{search}}(n)$$

With expected linear construction and expected constant-time searching:

$$T_{\text{total}}(n, q) = O(n) + O(q)$$

For $q$searches, this is expected $O(n + q)$total time, excluding database loading and updates. If the hash table must be rebuilt for every search, the preprocessing cost would become much more expensive. Therefore, the table should normally be maintained and reused when the database changes infrequently.

Interpretation: Hash Table Search is suitable for systems that perform many lookups because the expected search time does not grow proportionally with the number of stored records. Its performance depends on the hash function, table size, collision-resolution strategy, and load factor. For the university system, the table construction cost can be spread across many searches because new records are added only occasionally.

3.5 Mathematical Comparison of the Three Algorithms

Input size (n)	**Linear Search: worst-case comparisons**	**Binary Search: approximate halving steps**	**Hash Table Search: expected search behaviour**
1,000	1,000	10	Approximately constant
10,000	10,000	14	Approximately constant
100,000	100,000	17	Approximately constant
1,000,000	1,000,000	20	Approximately constant

The table illustrates the difference between the three approaches. Linear Search examines an increasing number of records as the database grows. Binary Search increases the number of steps slowly because it halves the search range repeatedly. Hash Table Search is expected to require a small number of operations under suitable hashing conditions, although actual performance depends on collisions and table design.

3.6 Effect of Preprocessing on Total Efficiency

Preprocessing must be included when comparing the algorithms fairly. Linear Search can begin searching immediately without sorting. Binary Search requires sorted records, and sorting an unsorted dataset may cost $O(n \log n)$. Hash Table Search requires constructing a table, which generally takes expected $O(n)$time with a suitable implementation.

For a single search, the preprocessing cost can be a significant part of the total execution time. For thousands of repeated searches, preprocessing can be shared across many requests, making Binary Search or Hash Table Search more attractive. The university receives new records only once per day under the changed requirement, so maintaining a sorted array or a hash table may be practical, depending on the update and storage requirements.

3.7 Conclusion

The mathematical analysis shows that Linear Search has linear search time, Recursive Binary Search has logarithmic search time on sorted data, and Hash Table Search has expected constant-time search under suitable hashing conditions. The total cost must include sorting or table construction where applicable. These results provide a theoretical foundation for comparing best-, average-, and worst-case behaviour in the next task.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour

4.1 Introduction

Algorithm performance depends not only on the size of the input but also on the position of the required student registration number, the arrangement of the records, and the behaviour of the search method. Best-case analysis describes the minimum work performed by an algorithm, average-case analysis describes the expected work under stated assumptions, and worst-case analysis describes the maximum work required. In this task, Linear Search, Recursive Binary Search, and Hash Table Search are analysed for the university student registration database. The database size is represented by n, where $n$is the total number of student records.

4.2 Linear Search

Linear Search examines student registration numbers one by one, starting from the first record. It stops when the required registration number is found or when all records have been checked. **A. Best-Case Behaviour** The best case occurs when the required registration number is stored in the first position. The algorithm performs only one comparison and stops immediately. Number of comparisons: C(n) = 1 Time complexity: Ω(1) Example: If the database contains 1,000,000 records and the required registration number is the first record, Linear Search finds it after one comparison. **B. Average-Case Behaviour** The average case occurs when the required registration number is equally likely to appear at any position in the database. Assuming the number is present, the algorithm may examine the first record, the second record, and so on, up to the last record. Average number of comparisons: C(n) = (1 + 2 + 3 + ... + n) / n = (n + 1) / 2 Therefore, the average number of comparisons increases proportionally with the number of records. The average time complexity is Θ(n). Example: For 1,000 records, the average is 500.5 comparisons. For 1,000,000 records, the average is 500,000.5 comparisons. **C. Worst-Case Behaviour** The worst case occurs when the required registration number is in the last position or is not present in the database. In both situations, the algorithm may examine all $n$records. Number of comparisons: C(n) = n Time complexity:

O(n) Example: For 1,000,000 records, Linear Search may perform 1,000,000 comparisons before completing the search. **D. Discussion** Linear Search is simple and does not require the records to be sorted. However, its performance decreases as the database grows because it may need to examine a large number of records for each search.

4.3 Recursive Binary Search

Recursive Binary Search works on a sorted list of registration numbers. It compares the required number with the middle element and continues searching only the half that may contain the target. **A. Best-Case Behaviour** The best case occurs when the required registration number is exactly the middle element in the first comparison. The algorithm finds the target immediately. Number of comparisons: C(n) = 1 Time complexity: Ω(1) Example: If the middle record of a sorted database contains the required registration number, the search ends after one comparison, even if the database contains one million records. **B. Average-Case Behaviour** In the average case, the required registration number is found after repeatedly reducing the search range by approximately half. The number of comparisons grows logarithmically with the number of records. Approximate number of comparisons: $C(n) \approx \log_2(n)$ Time complexity: Θ(log n) For example, a database of 1,000 records requires about 10 halving steps, 10,000 records about 14, 100,000 records about 17, and 1,000,000 records about 20. The exact number depends on the position of the target and the implementation. **C. Worst-Case Behaviour** The worst case occurs when the target is found only after the search range has been reduced repeatedly to a single element, or when the target is absent and the algorithm finishes checking the possible range. After $k$steps, the remaining search range is approximately $n/2^k$. The search reaches a range of one element when $n/2^k \leq 1$, which gives $k \geq \log_2 n$. Therefore, the worst-case time complexity is O(log n). For a database of 1,000,000 records, approximately 20 halving steps are sufficient to reduce the search range to one element. **D. Discussion** Recursive Binary Search performs very few comparisons for large sorted databases. However, the registration numbers must be sorted before searching. If sorting is required, its cost must be considered separately, especially when the database changes frequently.

4.4 Hash Table Search

Hash Table Search uses a hash function to map a registration number to a position in a hash table. It can provide very fast access when the hash function distributes keys well and collisions are handled efficiently. **A. Best-Case Behaviour** The best case occurs when the hash function maps the required registration number directly to an available position and no collision needs to be handled. Number of comparisons: approximately 1 Time complexity: Ω(1) The algorithm can access the required record without examining many other registration numbers. **B. Average-Case Behaviour** The average case occurs when the hash function distributes registration numbers reasonably evenly and the table maintains a suitable load factor. The required record can usually be located with a small, nearly constant amount of work. Expected time complexity: Θ(1) For example, searching among 1,000 records or 1,000,000 records may require only a small number of operations on average, assuming the table is appropriately designed and maintained. This is an expected result, not a guarantee for

every search. **C. Worst-Case Behaviour** The worst case occurs when many registration numbers map to the same position or collide heavily. Depending on the collision-resolution method, the search may need to examine many entries. Worst-case time complexity: O(n) If many records share the same hash location, the algorithm may perform a linear number of comparisons. Therefore, constant-time search is not guaranteed for every possible input. **D. Discussion** Hash Table Search is suitable for applications that perform many exact registration-number searches. It requires memory for the table and an appropriate collision-handling method. Adding new records may also require resizing or reorganising the table when its capacity becomes insufficient.

4.5 Comparison of Best, Average and Worst Cases

Algorithm	Best Case	Average Case	Worst Case
Linear Search	Θ(1)	Θ(n)	O(n)
Recursive Binary Search	Θ(1)	Θ(log n)	O(log n)
Hash Table Search	Θ(1)	Expected Θ(1)	O(n)

The table shows that Linear Search has the simplest requirements but may take a long time for large databases. Recursive Binary Search provides predictable logarithmic search performance when records are sorted. Hash Table Search offers expected constant-time access when the hash table is designed and maintained properly, but its worst case can be linear.

4.6 Comparison for Different Database Sizes

Number of Records	Linear Search: Worst-Case Comparisons	Binary Search: Approximate Halving Steps	Hash Table Search: Expected Behaviour
1,000	1,000	10	Nearly constant work
10,000	10,000	14	Nearly constant work
100,000	100,000	17	Nearly constant work
1,000,000	1,000,000	20	Nearly constant work

These figures are theoretical estimates used to compare growth patterns, not measured execution times. Actual performance depends on the implementation, hardware, data arrangement, and hash-table design.

4.7 Importance of Case Analysis in the Student Registration System

The university system must retrieve student records as the database grows from 1,000 to 1,000,000 or more records. Best-case performance alone is not sufficient to select an algorithm because a registration number may appear anywhere in the database or may not exist. Linear Search can be acceptable for small or unsorted collections, but the number of comparisons can become very high for large datasets. Recursive Binary Search is more suitable when the registration numbers are sorted and the sorted order can be maintained. Hash Table Search is a strong option when exact registration-number lookups are frequent and the system can maintain a well-designed hash table. The system must also consider preprocessing, memory usage, and record updates. Sorting the records or building a hash table involves additional work, but this work can be worthwhile when many searches are performed on the same database.

4.8 Conclusion

Best-, average-, and worst-case analysis helps evaluate algorithms under different search conditions. Linear Search has constant best-case time but linear average- and worst-case time. Recursive Binary Search has constant best-case time and logarithmic average- and worst-case time when the data is sorted. Hash Table Search has expected constant-time performance with effective hashing, but its worst-case time can be linear. For a large university registration database, the final choice should depend on search frequency, whether records are sorted, update requirements, memory availability, and experimental results. These factors will be considered in the following tasks.

TASK 5 – Perform Asymptotic Analysis

5.1 Introduction

Asymptotic analysis is used to study how the running time and number of operations of an algorithm increase when the input size becomes larger. It helps compare algorithms without depending on a particular computer, programming language, or hardware configuration. In the university student registration system, the number of records increases from 1,000 to 10,000, 100,000, and 1,000,000 or more. Asymptotic analysis helps determine which searching algorithm is suitable for this growth. The three algorithms considered are Linear Search, Recursive Binary Search, and Hash Table Search. The main asymptotic notations used are Big-O, Big-Omega, and Big-Theta.

5.2 Big-O Notation

Big-O notation represents an asymptotic upper bound on the growth of an algorithm's running time. It is commonly used to describe worst-case time complexity. **A. Linear Search** In the worst case, Linear Search examines every record before finding the registration number or determining that it is absent. Number of comparisons: C(n) = n Time complexity: O(n) As the database grows, the number of comparisons can increase in direct proportion to the number of records. If the number of records increases from 1,000 to 1,000,000, the worst-case

number of comparisons increases from 1,000 to 1,000,000. **B. Recursive Binary Search** Recursive Binary Search reduces the search range by approximately half at each step. Its worst-case number of comparisons grows logarithmically. Time complexity: O(log n) For one million records, approximately 20 halving steps are needed to reduce the search range to one element. Therefore, Binary Search is more efficient than Linear Search for large, sorted datasets. **C. Hash Table Search** With a suitable hash function and effective collision handling, Hash Table Search usually takes a nearly constant amount of time. Expected time complexity: O(1) However, the worst-case time complexity is O(n) when many registration numbers collide and the algorithm must examine many entries.

5.3 Big-Omega Notation

Big-Omega notation represents an asymptotic lower bound on the growth of an algorithm's running time. It describes a lower bound on the amount of work required under the conditions being analysed. **A. Linear Search** The best case occurs when the required registration number is found in the first position. Number of comparisons: C(n) = 1 Best-case time complexity: Ω(1) **B. Recursive Binary Search** The best case occurs when the required registration number is found at the middle position during the first comparison. Number of comparisons: C(n) = 1 Best-case time complexity: Ω(1) **C. Hash Table Search** When the hash function directly identifies the required entry and no collision needs to be resolved, the search requires a constant amount of work. Best-case time complexity: Ω(1) These lower bounds show that all three algorithms can perform a search quickly under favourable conditions. However, best-case performance alone is not sufficient to determine which algorithm is most suitable for a large database.

5.4 Big-Theta Notation

Big-Theta notation represents a tight asymptotic bound. It describes the growth rate when the running time is bounded both above and below by constant multiples of the same function. **A. Linear Search** For an unsuccessful search, every record must be examined. Its running time therefore grows linearly with the input size. Time complexity: Θ(n) **B. Recursive Binary Search** For searches that require repeatedly reducing the search range, the number of comparisons grows logarithmically. Time complexity: Θ(log n) **C. Hash Table Search** Under suitable hashing assumptions, such as reasonably uniform key distribution and a controlled load factor, the expected search time is constant. Expected time complexity: Θ(1) The expected constant-time result depends on the hash-table implementation and input distribution. The worst case remains linear.

5.5 Comparison of Asymptotic Complexities

Algorithm	Best Case	Average / Expected Case	Worst Case
Linear Search	Θ(1)	Θ(n)	O(n)

Recursive Binary Search	Θ(1)	Θ(log n)	O(log n)
Hash Table Search	Θ(1)	Expected Θ(1)	O(n)

The table shows that Linear Search has linear average-case growth. Recursive Binary Search has logarithmic growth when the data is sorted. Hash Table Search offers expected constant-time searching under suitable conditions, but it can degrade to linear time in the worst case.

5.6 Growth Analysis for Different Database Sizes

Number of Records (n)	**Linear Search: Worst-Case Comparisons**	**Binary Search: Approximate Halving Steps**	**Hash Table Search**
1,000	1,000	10	Expected nearly constant work
10,000	10,000	14	Expected nearly constant work
100,000	100,000	17	Expected nearly constant work
1,000,000	1,000,000	20	Expected nearly constant work

These values are theoretical comparisons, not measured execution times. The Binary Search values represent approximate halving steps. Actual execution depends on the implementation, data arrangement, and hardware. The comparison demonstrates that Linear Search becomes increasingly expensive as the database grows. Binary Search increases its work very slowly because each comparison removes approximately half of the remaining search range. Hash Table Search can maintain fast expected access if the table is appropriately constructed and maintained.

5.7 Importance of Preprocessing

Asymptotic analysis must also consider the work required before searching begins. **Linear Search:** No sorting or table construction is required. The algorithm can search directly through the records. Its search complexity is O(n). **Recursive Binary Search:** The records must be sorted. If sorting is necessary, a comparison-based sorting algorithm such as Merge Sort typically requires O(n log n) time. Once sorting is complete, each Binary Search requires O(log n) time. If the database is already sorted and remains sorted, the sorting cost does not need to be repeated for every search. **Hash Table Search:** The hash table must be constructed

before searching. With a suitable hash function, building the table is expected to take O(n) time. After construction, each search is expected to take O(1) time. Insertions and resizing may also require additional work. Therefore, comparing only the search operation can be misleading. The total cost depends on the number of searches, preprocessing requirements, memory usage, and frequency of updates.

5.8 Application to the University Registration Database

For a small database, Linear Search may be acceptable because it is easy to implement and requires no preprocessing. For a database containing one million records, it may perform one million comparisons in the worst case. Recursive Binary Search is a suitable option when registration numbers are maintained in sorted order. Its logarithmic search complexity makes it effective for large datasets. Hash Table Search is particularly suitable when the system performs frequent exact searches using student registration numbers. Its expected constant-time performance can reduce the work required for repeated lookups, provided collisions and memory usage are managed properly. The university must also consider the expected number of searches and the frequency of new student registrations. These factors determine whether the preprocessing cost of sorting or building a hash table is worthwhile.

5.9 Conclusion

Asymptotic analysis provides a theoretical method for comparing the growth rates of searching algorithms. Linear Search has O(n) worst-case time, Recursive Binary Search has O(log n) worst-case time on sorted data, and Hash Table Search has expected O(1) search time with a worst case of O(n). For a university database that may contain one million or more records, Binary Search and Hash Table Search generally provide better search performance than Linear Search under their required conditions. The final decision must also consider preprocessing, updates, memory usage, and experimental results.

TASK 6 – Formulate the Recurrence Relation

6.1 Introduction

A recurrence relation is a mathematical equation that describes the running time of a recursive algorithm in terms of the running time of a smaller input. It explains how the work performed on the current problem is related to the work performed on its subproblems. In this assignment, Linear Search, Recursive Binary Search, and Hash Table Search are considered for searching student registration numbers. Among these three approaches, **Recursive Binary Search is selected for recurrence-relation analysis** because it solves the problem recursively by reducing the search range into a smaller subproblem.

6.2 Identification of the Recursive Algorithm

Recursive Binary Search searches for a registration number in a sorted list. It compares the target with the middle element of the current search range. If the target is not found, the

algorithm calls itself on either the left half or the right half of the range. The algorithm follows these steps:

1. Check whether the current search range is empty. If it is empty, return that the registration number is not found.
2. Calculate the middle position of the current range.
3. Compare the middle element with the required registration number.
4. If both are equal, return the position of the registration number.
5. If the target is smaller than the middle element, recursively search the left half.
6. If the target is greater than the middle element, recursively search the right half.
7. Continue until the registration number is found or the search range becomes empty.

6.3 Identification of the Input Size

Let $n$represent the number of student registration records in the current search range. Initially, the algorithm may search all $n$records. After each comparison, it continues with approximately half of the previous search range. Therefore, the next recursive call operates on approximately $n/2$records. For example, if the database contains 1,000,000 sorted records, the search range is reduced approximately as follows: 1,000,000 → 500,000 → 250,000 → 125,000 → 62,500, and so on, until the target is found or no possible records remain.

6.4 Work Performed Outside the Recursive Call

Before making a recursive call, Binary Search performs a constant amount of work:

- Checks whether the search range is empty.
- Calculates the middle position.
- Compares the middle registration number with the target.
- Determines which half should be searched next. These operations require constant time, represented by c, where $c$is a constant independent of the input size. The algorithm makes only one recursive call at each step because it searches either the left half or the right half, not both halves.

6.5 Formulation of the Recurrence Relation

Let $T(n)$represent the running time of Recursive Binary Search for an input of size n. At each step, the algorithm performs constant work and then searches approximately half of the current range. Therefore, the recurrence relation is: **T(n) = T(n/2) + c** where:

- $T(n)$is the time required to search $n$records.
- $T(n/2)$is the time required to search approximately half of the records.

- $c$represents the constant work performed at the current step.
- $n$is the number of records in the current search range. This relation assumes that the input is sorted and that the algorithm continues with one half of the range.

6.6 Base Case

A recursive algorithm must have a base case to stop further recursive calls. In Recursive Binary Search, the base case occurs when the search range contains one record or becomes empty. For a range of one record, the algorithm performs a comparison and returns the result. For an empty range, it returns that the target is not present. The base case can be represented as: **T(1) = d** where $d$is a constant representing the work performed for the base case. An empty search range also takes constant time. The base case prevents infinite recursion and allows the algorithm to terminate correctly.

6.7 Number of Recursive Calls

At each recursive call, the search range is reduced by approximately half. After one step, the range is $n/2$; after two steps, it is $n/4$; after three steps, it is $n/8$. After $k$steps, the remaining range is approximately $n/2^k$. The recursion stops when the range contains at most one record. Therefore: $n/2^k \leq 1$Rearranging gives: $n \leq 2^k$Taking logarithms to base 2 gives: $\mathbf{k \geq \log_2(n)}$ Thus, the number of recursive steps grows logarithmically with the number of records.

6.8 Recurrence Relation for Different Input Sizes

The recurrence relation can be applied to different database sizes.

Input Size	First Reduced Search Range	Approximate Halving Steps
1,000	500	10
10,000	5,000	14
100,000	50,000	17
1,000,000	500,000	20

The number of halving steps grows slowly compared with the number of records. These are theoretical estimates; the exact number of comparisons depends on the position of the target and the implementation.

6.9 Recurrence Relations for the Other Approaches

A. Linear Search Linear Search is iterative in the selected design, so a recursive recurrence is not required for its implementation. Its running time can be described using the loop operation count. In the worst case, it examines all $n$records, giving linear time complexity. If

Linear Search were implemented recursively, its recurrence could be written as $T(n) = T(n-1) + c$, with a constant-time base case. However, this is not the iterative version selected for this assignment. **B. Hash Table Search** Hash Table Search is based on computing a hash value and accessing the corresponding table location. Under suitable hashing conditions, the expected search work is constant. It does not require the same halving recurrence as Recursive Binary Search. Its worst-case search can become linear when many entries collide.

6.10 Importance of the Recurrence Relation

The recurrence relation explains why Recursive Binary Search performs efficiently on large, sorted databases. Instead of examining every registration number, it reduces the number of possible records by approximately half at every step. For the university database containing one million records, the number of halving steps is approximately 20. This makes Binary Search suitable when the registration numbers are sorted and the system needs to perform repeated searches. The recurrence also provides the mathematical foundation for the next task, where the relation $T(n) = T(n/2) + c$will be solved to determine the overall time complexity.

6.11 Conclusion

The recurrence relation for Recursive Binary Search is **T(n) = T(n/2) + c**, with the base case $T(1) = d$. The algorithm performs constant work at each step and continues searching in approximately half of the remaining range. The resulting number of recursive steps grows logarithmically with the input size. This recurrence relation will be solved in Task 7 to establish the time complexity mathematically.

TASK 7 – Solve the Recurrence Relation

7.1 Introduction

Solving a recurrence relation helps determine the time complexity of a recursive algorithm. A recurrence relation expresses the running time of an algorithm in terms of the running time for a smaller input. In Task 6, the recurrence relation for Recursive Binary Search was formulated as T(n) = T(n/2) + c, where $n$is the number of student registration records and $c$represents the constant work performed at each step. This task solves the recurrence relation using the **iteration method** and interprets the result for the university student registration database.

7.2 Recurrence Relation

The recurrence relation is: T(n) = T(n/2) + c Base case: T(1) = d Here, T(n) represents the time required to search $n$records, T(n/2) represents the time required to search approximately half of the current range, and $c$represents the constant work performed during each recursive call. The constant $d$represents the time required for the base case.

7.3 Solving Using the Iteration Method

The iteration method repeatedly expands the recurrence relation until the base case is reached. **Step 1: Write the original recurrence** T(n) = T(n/2) + c **Step 2: Expand the recurrence once** Substitute $n/2$for n: T(n/2) = T(n/4) + c Therefore: T(n) = T(n/4) + 2c **Step 3: Expand the recurrence again** T(n/4) = T(n/8) + c Therefore: T(n) = T(n/8) + 3c **Step 4: Identify the pattern** After three expansions, the recurrence contains T(n/8) and three constant-work terms. Continuing this process gives: T(n) = T(n/2^k) + kc where $k$is the number of recursive steps. **Step 5: Reach the base case** The recursion stops when the remaining search range is approximately one record. n/2^k = 1 Therefore: 2^k = n Taking logarithms to base 2: k = $\log_2(n)$ **Step 6: Substitute the value of k** Using T(n) = T(n/2^k) + kc and k = $\log_2(n)$: T(n) = T(1) + c $\log_2(n)$ Since T(1) = d: T(n) = d + c $\log_2(n)$ Therefore, the running time grows logarithmically with the input size. **Final time complexity: T(n) = Θ(log n)**

7.4 Verification Using the Master Theorem

The recurrence relation can also be solved using the Master Theorem: T(n) = T(n/2) + c The standard form of the Master Theorem is T(n) = aT(n/b) + f(n). For this recurrence:

- a = 1, because there is one recursive call.
- b = 2, because the problem size is reduced by half.
- f(n) = Θ(1), because the work outside the recursive call is constant. Calculate: n^(log_b a) = n^($\log_2$ 1) = n^0 = 1 Therefore, f(n) = Θ(1), which matches n^(log_b a) = 1. This is the first case of the Master Theorem. Hence: T(n) = Θ(log n) Both the iteration method and the Master Theorem produce the same result.

7.5 Interpretation of the Result

The solution T(n) = Θ(log n) means that the running time increases slowly as the number of student registration records increases. Unlike Linear Search, which may examine every record, Recursive Binary Search reduces the remaining search range by approximately half after each comparison. For example, when the database contains 1,000,000 records, approximately 20 halving steps are sufficient to reduce the search range to one record. Therefore, Binary Search is highly efficient for large, sorted databases.

7.6 Comparison with Linear Search and Hash Table Search

Algorithm	**Search Complexity**	**Explanation**
Linear Search	Θ(n) average and worst case	Examines records one by one
Recursive Binary Search	Θ(log n) average and worst case	Reduces the search range by half

Hash Table Search	Expected $\Theta(1)$; worst-case $O(n)$	Uses a hash function to locate a record

The comparison shows that Recursive Binary Search performs substantially fewer operations than Linear Search for large, sorted datasets. Hash Table Search may offer even faster expected lookup, but its performance depends on the hash function, collision handling, and table design.

7.7 Effect of Database Size

Number of Records	**Approximate $\log_2(n)$**	**Expected Binary Search Behaviour**
1,000	10	About 10 halving steps
10,000	13.3	About 14 halving steps
100,000	16.6	About 17 halving steps
1,000,000	19.9	About 20 halving steps

These values are theoretical estimates. The exact number of comparisons depends on the target's position, whether the target exists, and the implementation.

7.8 Practical Significance for the University Database

The university database may contain one million or more student records. If registration numbers are maintained in sorted order, Recursive Binary Search can locate a record with approximately logarithmic growth in the number of comparisons. However, sorting the records requires additional preprocessing if the data is not already sorted. If many searches are performed on the same sorted database, the sorting cost can be shared across those searches. If records are frequently inserted, the cost of maintaining sorted order must also be considered. For this reason, the theoretical result should be combined with implementation and experimental testing before selecting the final searching strategy.

7.9 Conclusion

The recurrence relation $T(n) = T(n/2) + c$ was solved using the iteration method and verified using the Master Theorem. Both methods produce the result $T(n) = \Theta(\log n)$. This confirms that Recursive Binary Search has logarithmic time complexity when searching sorted data. The result supports its use for large university registration databases, while preprocessing and update costs must also be considered in the final design.

TASK 8 – Implement and Test the Algorithms

8.1 Introduction

Implementation and testing are important steps in evaluating the performance and correctness of searching algorithms. The theoretical analysis explains how an algorithm grows with input size, while implementation helps verify whether the algorithm produces the expected results. In this task, three searching algorithms are implemented using Python: Linear Search, Recursive Binary Search, and Hash Table Search. The algorithms are tested using student registration numbers with different dataset sizes: 1,000, 10,000, 100,000, and 1,000,000 records. The main objectives are to verify correctness, compare searching methods, and prepare for experimental performance measurement.

8.2 Software and Testing Environment

The algorithms can be implemented using Python 3 in IDLE, VS Code, or Google Colab.

- Programming language: Python 3
- Input data: Student registration numbers
- Dataset sizes: 1,000, 10,000, 100,000, and 1,000,000
- Testing methods: Successful and unsuccessful searches
- Performance measures: Number of comparisons and execution time The same dataset and target values should be used wherever possible to make the comparison fair.

8.3 Algorithm 1: Linear Search

Description: Linear Search checks each registration number from the beginning of the list until the target is found or all records have been examined. **Python Program:**

Python

Run

```
def linear_search(data, target):
    comparisons = 0
    for i in range(len(data)):
        comparisons += 1
        if data[i] == target:
            return i, comparisons
    return -1, comparisons
```

Explanation: The function receives a list of registration numbers and a target value. It compares the target with each element. If a match is found, it returns the index and comparison count. If no match is found, it returns -1 and the total number of comparisons.

Expected Behaviour: The best case requires one comparison. The worst case requires $n$comparisons when the target is the last element or is absent.

8.4 Algorithm 2: Recursive Binary Search

Description: Recursive Binary Search works on a sorted list. It checks the middle element and recursively searches the appropriate half. **Python Program:**

Python

Run

```
def binary_search(data, target, low, high, comparisons=None):
    if comparisons is None:
        comparisons = [0]

    if low > high:
        return -1, comparisons[0]

    mid = (low + high) // 2
    comparisons[0] += 1

    if data[mid] == target:
        return mid, comparisons[0]
    elif target < data[mid]:
        return binary_search(data, target, low, mid - 1, comparisons)
    else:
        return binary_search(data, target, mid + 1, high, comparisons)
```

Explanation: The function calculates the middle index and compares its value with the target. If they are equal, it returns the index. Otherwise, it recursively searches either the left half or the right half. The comparison count is shared across recursive calls. **Expected Behaviour:** Binary Search requires sorted data. Its search complexity is logarithmic because the search range is reduced by approximately half at every recursive call.

8.5 Algorithm 3: Hash Table Search

Description: A Python dictionary can be used as a hash table. It maps each registration number to its associated record or position. **Python Program:**

Python

Run

```
def build_hash_table(data):
    return {value: i for i, value in enumerate(data)}


def hash_search(table, target):
    return table.get(target, -1)
```

Explanation: The build_hash_table function creates a dictionary containing each registration number and its index. The hash_search function retrieves the stored index. If the registration number does not exist, it returns -1. **Expected Behaviour:** Dictionary lookup has expected O(1) time complexity under suitable hashing conditions. Building the dictionary takes expected O(n) time and additional memory.

8.6 Complete Testing Program

The following program tests all three approaches on the four required dataset sizes. It measures execution time and records comparison counts for Linear Search and Binary Search. Hash Table construction and lookup are timed separately so that preprocessing is not confused with search time.

Python

Run

```
import time


def linear_search(data, target):
    comparisons = 0
    for i, value in enumerate(data):
        comparisons += 1
        if value == target:
            return i, comparisons
    return -1, comparisons


def binary_search(data, target, low, high, comparisons=None):
    if comparisons is None:
```

```
        comparisons = [0]

    if low > high:
        return -1, comparisons[0]

    mid = (low + high) // 2
    comparisons[0] += 1

    if data[mid] == target:
        return mid, comparisons[0]
    if target < data[mid]:
        return binary_search(data, target, low, mid - 1, comparisons)
    return binary_search(data, target, mid + 1, high, comparisons)

def build_hash_table(data):
    return {value: i for i, value in enumerate(data)}

def hash_search(table, target):
    return table.get(target, -1)

sizes = [1000, 10000, 100000, 1000000]

print("Size | Algorithm | Found Index | Comparisons | Time (seconds)")

for n in sizes:
    data = list(range(1, n + 1))
    target = n

    start = time.perf_counter()
```

```
    index, comparisons = linear_search(data, target)
    elapsed = time.perf_counter() - start
    print(n, "| Linear Search |", index, "|", comparisons, "|", round(elapsed, 6))

    start = time.perf_counter()
    index, comparisons = binary_search(data, target, 0, n - 1)
    elapsed = time.perf_counter() - start
    print(n, "| Binary Search |", index, "|", comparisons, "|", round(elapsed, 6))

    start = time.perf_counter()
    table = build_hash_table(data)
    build_time = time.perf_counter() - start

    start = time.perf_counter()
    index = hash_search(table, target)
    search_time = time.perf_counter() - start
    print(n, "| Hash Search |", index, "| N/A", "|", round(search_time, 6))
    print("Hash construction time:", round(build_time, 6), "seconds")
```

Note: The program uses sorted integer registration numbers for a controlled test. It searches for the last registration number, which produces the worst case for Linear Search. Binary Search and Hash Table Search use the same target. Execution times vary by computer and should be recorded from the actual run.

8.7 Test Cases

The following test cases should be used to verify the correctness of the algorithms.

Test Case	Input Condition	Expected Result
Test 1	Target is the first record	Correct index returned
Test 2	Target is the middle record	Correct index returned
Test 3	Target is the last record	Correct index returned

Test 4	Target is not present	-1 returned
Test 5	Dataset contains 1,000 records	Correct search result
Test 6	Dataset contains 10,000 records	Correct search result
Test 7	Dataset contains 100,000 records	Correct search result
Test 8	Dataset contains 1,000,000 records	Correct search result

8.8 Correctness Verification

The algorithms are considered correct when they return the correct index for a registration number that exists and return -1 when it does not exist. Linear Search checks records sequentially. Binary Search returns the correct result when the input is sorted. Hash Table Search returns the stored index using the registration number as the key. The test results should be checked against the original list to ensure that each returned index contains the expected registration number.

8.9 Testing Considerations

For a fair comparison, the following conditions should be maintained:

1. Use the same dataset sizes for all algorithms.
2. Test successful and unsuccessful searches.
3. Use sorted data for Binary Search.
4. Measure hash-table construction time separately from lookup time.
5. Repeat timing tests several times and calculate an average.
6. Avoid comparing one algorithm's preprocessing time with another algorithm's search-only time.
7. Record actual execution times instead of assuming that theoretical estimates are measured results. For the one-million-record dataset, the program may take longer to execute because Linear Search can examine every element. This is expected behaviour and is useful for demonstrating the difference between linear and logarithmic growth.

8.10 Expected Outcomes

Algorithm	**Correctness Requirement**	**Expected Performance**

Linear Search	Finds the target or returns -1	Work grows linearly in the worst case
Recursive Binary Search	Finds the target in sorted data or returns -1	Work grows logarithmically
Hash Table Search	Returns the stored index or -1	Expected constant-time lookup after construction

These are theoretical expectations. Actual execution-time values must be obtained by running the program.

8.11 Conclusion

Three searching algorithms have been implemented in Python and a testing procedure has been prepared for dataset sizes ranging from 1,000 to 1,000,000 records. The program checks correctness, counts comparisons where applicable, and measures execution time. The implementation provides the foundation for recording experimental performance in Task 9 and comparing actual behaviour with theoretical time complexity in Task 10.

TASK 9 – Record Experimental Performance

9.1 Introduction

Experimental performance analysis is used to evaluate the actual behaviour of searching algorithms by executing them with different input sizes. Theoretical analysis explains how the number of operations grows, while experimental analysis measures the execution time on a real computer. In this task, Linear Search, Recursive Binary Search, and Hash Table Search are evaluated using student registration databases containing 1,000, 10,000, 100,000, and 1,000,000 records. The main objective is to record the number of comparisons, execution time, and preprocessing time where applicable. These measurements help determine how efficiently each algorithm handles increasing database sizes.

9.2 Experimental Setup

The algorithms are implemented in Python 3 and executed using the same computer and software environment.

- **Programming language:** Python 3
- **Dataset:** Integer student registration numbers
- **Input sizes:** 1,000, 10,000, 100,000, and 1,000,000 records
- **Linear Search:** Searches records sequentially
- **Recursive Binary Search:** Searches a sorted list

- **Hash Table Search:** Uses a Python dictionary
- **Timing function:** time.perf_counter()
- **Performance measures:** Execution time, comparison count, and hash-table construction time To maintain a fair comparison, all algorithms should use the same target registration number for each dataset. The target should be selected deliberately to demonstrate different search conditions.

9.3 Performance Measures

A. Number of Comparisons The number of comparisons indicates how many times an algorithm compares a registration number with the target. Linear Search may perform $n$comparisons in the worst case. Binary Search performs approximately $\log_2 n$comparisons. Hash Table Search does not use the same sequential comparison count, so its lookup time and construction time should be measured separately. **B. Execution Time** Execution time is the amount of time required for an algorithm to complete a search. It is measured in seconds using Python's time.perf_counter() function. **C. Preprocessing Time** Preprocessing time is the time required to prepare data before searching. Binary Search may require sorting, while Hash Table Search requires building the hash table. These costs should be recorded separately from lookup time. **D. Average Execution Time** To reduce the effect of temporary system activity, each search can be repeated several times. The average time is calculated as: Average time = Total time for all repetitions / Number of repetitions For more reliable results, repeat the experiments under the same conditions and report the average measured time.

9.4 Experimental Data Collection

Run the Python program from Task 8 and record the actual results in the following tables. The tables are provided as templates because execution times depend on the computer and must not be invented. **Table 1: Linear Search Performance**

Number of Records	**Target Position**	**Comparisons**	**Execution Time (seconds)**
1,000	Last record	Record actual value	Record actual value
10,000	Last record	Record actual value	Record actual value
100,000	Last record	Record actual value	Record actual value

1,000,000	Last record	Record actual value	Record actual value

When the target is the last record, Linear Search examines every record. Therefore, the comparison count should equal the number of records. **Table 2: Recursive Binary Search Performance**

Number of Records	**Target Position**	**Comparisons**	**Execution Time (seconds)**
1,000	Last record	Record actual value	Record actual value
10,000	Last record	Record actual value	Record actual value
100,000	Last record	Record actual value	Record actual value
1,000,000	Last record	Record actual value	Record actual value

Binary Search operates on sorted data. Although the target is the last record, the middle element is checked first, so the number of comparisons remains logarithmic. **Table 3: Hash Table Search Performance**

Number of Records	**Construction Time (seconds)**	**Lookup Time (seconds)**	**Search Result**
1,000	Record actual value	Record actual value	Verify result
10,000	Record actual value	Record actual value	Verify result
100,000	Record actual value	Record actual value	Verify result
1,000,000	Record actual value	Record actual value	Verify result

Hash-table construction and lookup should be measured separately. This makes it possible to distinguish the initial cost of building the table from the cost of searching it.

9.5 Theoretical Reference for Experimental Results

The following table gives theoretical expectations to compare with the measured data. These are not actual experimental measurements.

Number of Records	Linear Search: Worst-Case Comparisons	Binary Search: Approximate Halving Steps	Hash Table Search
1,000	1,000	10	Expected constant-time lookup
10,000	10,000	14	Expected constant-time lookup
100,000	100,000	17	Expected constant-time lookup
1,000,000	1,000,000	20	Expected constant-time lookup

This theoretical reference shows that Linear Search performs increasingly more comparisons as the database grows. Binary Search increases its comparisons slowly, while Hash Table Search is expected to provide fast lookup after construction under suitable hashing conditions.

9.6 Graphical Representation

A graph can be created using the measured execution times to show how each algorithm behaves as the input size increases.

- **X-axis:** Number of student registration records.
- **Y-axis:** Measured execution time in seconds.
- **Lines:** Linear Search, Recursive Binary Search, and Hash Table Search. A logarithmic scale on the X-axis can help display all four dataset sizes clearly. Alternatively, a separate graph can be used for comparison counts, since comparison counts are easier to interpret independently of computer speed. The graph should be generated from the actual measurements collected during execution. It should not use theoretical estimates as if they were measured values.

9.7 Factors Affecting Experimental Performance

Several factors can influence the measured execution time:

1. **Hardware:** Processor speed and available memory affect program execution.
2. **Programming language:** Python's implementation and function-call overhead affect timing.
3. **Dataset size:** Larger datasets may require more processing and memory.
4. **Target position:** Linear Search performs fewer comparisons when the target is near the beginning.
5. **Sorting:** Binary Search requires sorted input, and sorting adds preprocessing cost when the input is not already sorted.
6. **Hash-table construction:** Building a dictionary takes additional time and memory.
7. **System activity:** Other programs running on the computer can affect timing measurements.
8. **Repeated execution:** Multiple runs and averaging help reduce the effect of temporary variations.

9.8 Interpretation of Expected Behaviour

Linear Search is expected to show a noticeable increase in execution time as the number of records grows, especially when the target is at the end or absent. Recursive Binary Search is expected to remain efficient for large sorted datasets because the number of comparisons grows logarithmically. Increasing the dataset from 1,000 to 1,000,000 records does not increase the number of comparisons by the same factor as Linear Search. Hash Table Search is expected to provide fast lookup after construction. However, the total cost must include table construction when evaluating a database that is searched only a small number of times. These are expected patterns based on algorithmic complexity. The actual results must be examined before drawing conclusions about measured performance.

9.9 Conclusion

Experimental performance analysis provides practical evidence about how the three searching algorithms behave as the student database grows. The execution time, comparison count, and preprocessing cost are the main measures used in this task. The tables and graph should be completed using results from the Python program. The measured data will be used in Task 10 to compare theoretical time complexity with experimental behaviour and explain any differences.

TASK 10 – Compare Theory with Experimental Behaviour

10.1 Introduction

Theoretical analysis predicts how the running time of an algorithm increases with input size, while experimental analysis measures its actual performance by executing the program. Comparing these two results helps determine whether the implemented algorithm behaves as

expected. In this assignment, Linear Search, Recursive Binary Search, and Hash Table Search are compared using student registration databases containing 1,000, 10,000, 100,000, and 1,000,000 records. The theoretical results are based on time complexity, while the experimental results are obtained from the Python programs developed in Task 8 and measured in Task 9.

10.2 Theoretical Performance

The theoretical time complexities of the three algorithms are as follows: **A. Linear Search** Linear Search examines records one by one until the target is found or the list ends. Its best-case time complexity is $\Theta(1)$, while its average and worst-case time complexities are $\Theta(n)$. When the target is at the end of the list or is absent, the algorithm may examine all records. Therefore, its work increases proportionally with the database size. **B. Recursive Binary Search** Recursive Binary Search compares the target with the middle element and continues with approximately half of the search range. Its best-case time complexity is $\Theta(1)$, while its average and worst-case time complexities are $\Theta(\log n)$, assuming sorted input. This means that the number of comparisons increases slowly even when the database becomes very large. **C. Hash Table Search** Hash Table Search uses a hash function to locate a record. Its expected search time is $\Theta(1)$ when hashing and collision handling are effective. Its worst-case time complexity is $O(n)$. The table must be constructed before searching, and this preprocessing cost should be considered when comparing total performance.

10.3 Theoretical Comparison Table

Number of Records	Linear Search: Worst-Case Comparisons	Binary Search: Approximate Halving Steps	Hash Table Search
1,000	1,000	10	Expected constant-time lookup
10,000	10,000	14	Expected constant-time lookup
100,000	100,000	17	Expected constant-time lookup
1,000,000	1,000,000	20	Expected constant-time lookup

These figures describe theoretical operation counts, not measured execution times. They provide a reference for interpreting the experimental results.

10.4 Experimental Performance Comparison

The actual execution times recorded in Task 9 should be entered in the following table.

Number of Records	Linear Search Time (s)	Binary Search Time (s)	Hash Lookup Time (s)
1,000	Record measured value	Record measured value	Record measured value
10,000	Record measured value	Record measured value	Record measured value
100,000	Record measured value	Record measured value	Record measured value
1,000,000	Record measured value	Record measured value	Record measured value

The measured values must come from running the programs on the actual computer. Execution times should not be invented because they depend on the hardware, software environment, and implementation. For a fair comparison, the same target values and datasets should be used. Hash-table construction time should be reported separately from lookup time, and sorting time should be included when the original data is not already sorted.

10.5 Comparison of Theoretical and Experimental Behaviour

A. Linear Search Theoretical analysis predicts linear growth in the average and worst cases. Experimental results are expected to show an increase in execution time as the number of records increases, particularly when searching for the last record or a target that is absent. If the dataset grows from 1,000 to 1,000,000 records, the worst-case comparison count increases from 1,000 to 1,000,000. The measured time may not increase by exactly the same factor because of implementation and hardware effects. **B. Recursive Binary Search** Theoretical analysis predicts logarithmic growth. Experimental results are expected to show that search time increases much more slowly than Linear Search as the dataset grows. For example, the approximate number of halving steps increases from 10 for 1,000 records to 20 for 1,000,000 records. This demonstrates how Binary Search can handle large sorted datasets efficiently. **C. Hash Table Search** Theoretical analysis predicts expected constant-time lookup under suitable hashing conditions. Experimental lookup time may remain relatively low as the dataset grows, although table construction requires additional time and memory. Small timing differences between dataset sizes do not necessarily contradict constant-time

expected complexity. Real implementations have overheads, and actual behaviour depends on hash distribution, collisions, and memory access.

10.6 Reasons for Differences Between Theory and Experiment

The theoretical complexity and measured execution time may not match perfectly for several reasons.

1. **Hardware differences:** Processor speed and memory capacity affect execution time.
2. **Programming overhead:** Python function calls, object handling, and dictionary operations introduce overhead.
3. **Input arrangement:** Linear Search performance depends on the position of the target.
4. **Preprocessing costs:** Sorting and hash-table construction require additional time before searching.
5. **Memory usage:** Large datasets can affect memory allocation and access performance.
6. **Operating system activity:** Other running programs can affect timing measurements.
7. **Measurement overhead:** Very short operations may be difficult to measure accurately with a single timing run.
8. **Algorithm assumptions:** Binary Search requires sorted data, while expected Hash Table performance depends on effective hashing. These factors explain why theoretical complexity is useful for understanding growth but does not predict an exact execution time for every computer.

10.7 Importance of Repeated Experiments

A single timing measurement may not represent typical performance. To improve reliability, each search should be repeated several times, and the average execution time should be calculated. The experiments should use the same input conditions and target positions. If an algorithm executes too quickly for accurate timing, many searches can be measured together and the total time can be divided by the number of searches. Preprocessing and lookup times should be measured independently. This is especially important for Hash Table Search because building the table can take longer than an individual lookup.

10.8 Overall Comparison

Factor	**Linear Search**	**Recursive Binary Search**	**Hash Table Search**
Data requirement	Sorted order not required	Sorted data required	Hash table must be constructed

Search growth	Linear	Logarithmic	Expected constant
Extra memory	Low	Low for the list, plus recursion stack	Additional memory for the table
Preprocessing	Usually none	Sorting may be required	Table construction required
Suitable use	Small or unsorted collections	Large sorted collections	Frequent exact-key searches

Linear Search is simple and useful when the dataset is small or unsorted. Recursive Binary Search offers predictable performance when records are sorted. Hash Table Search is a strong option for frequent exact registration-number lookups when sufficient memory is available.

10.9 Conclusion

Theoretical analysis indicates that Linear Search has linear average and worst-case growth, Recursive Binary Search has logarithmic growth on sorted data, and Hash Table Search offers expected constant-time lookup under suitable conditions. Experimental testing is required to confirm how these differences appear on the actual computer. The measured results from Task 9 should be used to identify trends, explain any differences, and evaluate the overall cost of preprocessing. This comparison provides the evidence needed to recommend a suitable searching algorithm for the university registration database in Task 11.

TASK 11 – Recommend and Defend the Final Design

11.1 Introduction

Selecting the best searching algorithm is important for developing an efficient university student registration system. The system must search student registration numbers quickly, handle a large number of records, and support future database growth. In this assignment, three algorithms have been analysed: Linear Search, Recursive Binary Search, and Hash Table Search. Their theoretical complexity, preprocessing requirements, memory usage, and expected experimental performance are considered to recommend a suitable design. The university database is expected to grow from 1,000 records to 1,000,000 or more records. Therefore, the final design should provide efficient searching without unnecessary processing overhead.

11.2 Evaluation of the Three Algorithms

A. Linear Search Linear Search is simple to implement and does not require the records to be sorted. It examines each record sequentially until the required registration number is

found. Its best-case time complexity is Θ(1), while its average and worst-case time complexities are Θ(n). As the database grows, the number of comparisons can become very large. Advantages:

- Simple to implement and understand.
- Works with sorted and unsorted data.
- Requires very little additional memory. Limitations:
- Slow for large datasets in the average and worst cases.
- May examine one million records for a single unsuccessful search.
- Not ideal for systems that perform frequent searches. Linear Search is suitable for small datasets or situations where records cannot be sorted and searches are infrequent. **B. Recursive Binary Search** Recursive Binary Search reduces the search range by half after each comparison. It provides logarithmic search performance when the registration numbers are sorted. Its best-case time complexity is Θ(1), and its average and worst-case time complexities are Θ(log n). Advantages:
- Efficient for large, sorted datasets.
- Requires relatively few comparisons.
- Provides predictable worst-case search performance. Limitations:
- Requires sorted data.
- Sorting an unsorted database adds preprocessing time.
- Maintaining sorted order may add cost when new records are inserted. Recursive Binary Search is suitable when the database can be maintained in sorted order and efficient searching is required. **C. Hash Table Search** Hash Table Search uses a hash function to locate a registration number in a table. Under suitable hashing conditions, lookup takes expected constant time. Advantages:
- Very fast expected lookup for exact registration-number searches.
- Suitable for repeated searches.
- Does not require the registration numbers to be sorted. Limitations:
- Requires additional memory for the hash table.
- Building the table introduces preprocessing cost.
- Collisions must be handled properly.
- Worst-case search time can become O(n). Hash Table Search is suitable for applications that perform frequent exact-key searches and can maintain the table efficiently.

11.3 Comparison of the Algorithms

Evaluation Factor	Linear Search	Recursive Binary Search	Hash Table Search
Best-case time	$\Theta(1)$	$\Theta(1)$	Expected $\Theta(1)$
Average-case time	$\Theta(n)$	$\Theta(\log n)$	Expected $\Theta(1)$
Worst-case time	$O(n)$	$O(\log n)$	$O(n)$
Requires sorted data	No	Yes	No
Additional memory	Low	Low, plus recursion stack	Higher
Preprocessing	Usually none	Sorting may be required	Table construction
Large database suitability	Limited	High	High for exact lookups

The comparison indicates that Linear Search is less suitable for frequent searches in a very large database. Binary Search is efficient when the data is sorted. Hash Table Search is particularly useful when the system frequently retrieves records using exact registration numbers.

11.4 Recommended Final Design

Recommendation: Hash Table Search for frequent exact registration-number lookups. For the university registration system, a hash table is recommended as the primary search structure because the application needs to retrieve records using exact student registration numbers. Under suitable hashing conditions, the expected lookup time is $\Theta(1)$, which is efficient for a database containing one million or more records. The hash table should map each unique registration number to the corresponding student record or database identifier. The system should use an appropriate hash function, handle collisions correctly, monitor the load factor, and resize the table when necessary. If the university also needs to display registration numbers in sorted order or retrieve records by a range of registration numbers, a sorted index can be maintained separately. This allows the system to support different retrieval requirements without relying on a single searching method for every task.

11.5 Justification Based on Time Complexity

The recommended design is supported by the theoretical analysis:

- Linear Search has $\Theta(n)$ average and worst-case search time.
- Recursive Binary Search has $\Theta(\log n)$ average and worst-case search time when the records are sorted.
- Hash Table Search has expected $\Theta(1)$ lookup time under suitable hashing conditions. For one million records, Linear Search may examine up to one million entries. Binary Search requires approximately 20 halving steps in a typical worst-case comparison model. Hash Table Search is expected to locate the corresponding entry with a small amount of work, although the actual number of operations depends on implementation and collisions. These complexity results support using a hash table for frequent exact lookups. They do not guarantee a particular execution time, so the measured results from Task 9 should also be considered.

11.6 Justification Based on Experimental Performance

The experimental results should be used to confirm the recommendation. The measured execution times from Task 9 should be compared for all four dataset sizes. If Hash Table Search provides the lowest lookup time in the measured tests, this supports its selection for repeated exact-key searches. However, its construction time and additional memory must also be considered. If the database is searched only a few times, preprocessing may reduce its overall advantage. The measured values should be inserted into the comparison table before submitting the assignment. Since actual execution times depend on the computer and program implementation, theoretical estimates should not be presented as measured results.

11.7 Preprocessing and Update Considerations

The university registration database receives new student records over time. The final design must account for these updates. For the hash-table approach, a newly registered student's registration number can be added as a new key. If the table becomes too full, it may need to resize and reorganise its entries. Such resizing can require additional work, but lookups remain expected constant-time under suitable conditions. Binary Search requires sorted records. New registrations may need to be inserted into the correct sorted position or handled through a separate update strategy. Linear Search requires no sorting or index construction, but each search may be slower. Therefore, the hash-table design provides a practical balance between fast lookup and the ability to incorporate new records.

11.8 Final Design Proposal

The proposed design consists of the following steps:

1. Receive the student's registration number as the search key.
2. Validate the input registration number.
3. Use the hash function to locate the corresponding table entry.
4. Retrieve the associated student record or database identifier.
5. If the registration number is not found, display an appropriate message.

6. When a new student is registered, insert the new key and associated record.
7. Monitor table capacity and handle collisions and resizing appropriately.
8. Test the implementation regularly using different dataset sizes and search conditions. This design supports quick retrieval while allowing the database to grow. In a production university system, the hash table may be implemented as an in-memory index or replaced by an appropriate database index depending on persistence, concurrency, and reliability requirements.

11.9 Limitations and Risk Management

Although Hash Table Search is recommended, several limitations must be considered. First, additional memory is required to store the hash table. Second, poor hash distribution may increase collisions and reduce performance. Third, building the table takes time, especially for a large database. Finally, a production system must ensure that registration numbers are unique and that updates do not create inconsistent records. These risks can be reduced by choosing a reliable hash-table implementation, monitoring performance, validating registration numbers, and testing insertion and lookup operations. Experimental measurements should be repeated when the dataset or implementation changes.

11.10 Conclusion

After comparing Linear Search, Recursive Binary Search, and Hash Table Search, Hash Table Search is recommended for frequent exact registration-number lookups in a large university database. Its expected constant-time lookup makes it suitable for handling one million or more records, provided that hashing, collision handling, and memory usage are managed effectively. Recursive Binary Search remains useful when sorted order is required, while Linear Search is appropriate for small or unsorted collections. The final design should be confirmed using actual experimental results and adapted to the university's workload and update requirements.

TASK 12 – Re-evaluate and Adapt the Design

12.1 Introduction

The performance requirements of a system may change over time. An algorithm that works well for a small database may require modification when the number of records and search requests increases. In the university student registration system, the original database is expected to grow to one million or more records. The new challenge introduces a workload of **50,000 searches per hour on a database containing 1,000,000 records**, while new student records are added only once per day. The purpose of this task is to evaluate whether the original design remains suitable, analyse its limitations, and propose an improved design based on the updated requirements.

A. Impact Analysis

A.1 Impact of Increased Search Requests

The updated requirement specifies 50,000 searches per hour. This represents a high volume of search operations that must be handled efficiently. Average search rate = 50,000 / 3,600 ≈ 13.89 searches per second. The system must therefore support approximately 14 searches per second on average. Actual peak demand may be higher if many students or staff access the database at the same time. Linear Search is not suitable as the primary method for this workload because it may examine all one million records for a single unsuccessful search. Recursive Binary Search is more efficient because it reduces the search range by half at every step. For one million records, approximately 20 halving steps are sufficient to reduce the range to one record. Hash Table Search is also suitable because it provides expected constant-time lookup under suitable hashing conditions. This makes it a strong option for frequent exact registration-number searches.

A.2 Impact of Database Size

A database containing one million student records requires a searching method that avoids unnecessary scanning of the entire dataset. Linear Search has O(n) worst-case time complexity. As the number of records increases, the number of comparisons may increase proportionally. Recursive Binary Search has O(log n) worst-case time complexity, provided that the records are sorted. It performs relatively few comparisons even for large datasets. Hash Table Search has expected O(1) lookup time when the hash function and collision-handling strategy work effectively. However, its worst-case time complexity can become O(n).

A.3 Impact of Daily Record Updates

The updated scenario states that new records are added only once per day. This means that the database does not need to handle continuous record insertions throughout the day. This update pattern makes preprocessing more practical. The system can add the day's new records to the hash table or update its index before normal searching continues. A sorted index could also be maintained because updates are relatively infrequent. However, inserting records into an array while preserving sorted order may require shifting elements. A suitable database index or another data structure can help manage these updates.

A.4 Summary of the Impact

Requirement	**Impact on Design**
One million records	Requires efficient searching
50,000 searches per hour	Fast repeated lookup is important
Approximately 14 searches per second on average	The design must support the expected workload

New records added once per day	Daily preprocessing or index updates are practical
Exact registration-number searches	Hash Table Search is a strong candidate

The updated requirements favour an indexed searching design rather than scanning the entire database for every request.

B. Evaluation of Original Design

B.1 Original Design

The original design compared three searching algorithms: Linear Search, Recursive Binary Search, and Hash Table Search. Linear Search was selected as the simplest approach, Recursive Binary Search was considered for sorted records, and Hash Table Search was identified as a suitable method for frequent exact registration-number searches. The original recommendation was to use Hash Table Search as the primary lookup method.

B.2 Evaluation of Linear Search

Linear Search does not require sorting or additional indexing. However, it may examine one million records for a single unsuccessful search. If every one of the 50,000 hourly searches required scanning all one million records, the system could perform up to 50,000,000,000 record comparisons in one hour. This is a theoretical worst-case estimate, not a measured result. It demonstrates why Linear Search is unsuitable as the main strategy for this workload.

B.3 Evaluation of Recursive Binary Search

Recursive Binary Search performs approximately 20 halving steps for one million records in the worst-case comparison model. It is substantially more efficient than Linear Search for sorted data. However, the system must maintain sorted records or a sorted index. Since records are added only once per day, updating the sorted structure daily is practical, but the cost depends on the implementation. Binary Search remains a suitable alternative or supplementary method when sorted order is needed.

B.4 Evaluation of Hash Table Search

Hash Table Search is suitable for frequent exact-key searches because it provides expected O(1) lookup time under appropriate conditions. The daily update schedule allows the system to add newly registered students to the hash table in a controlled update process. The system must still manage collisions, capacity, and memory usage. The actual suitability should be verified through load testing using the expected search volume and dataset size.

B.5 Evaluation Result

The original recommendation remains appropriate in principle: Hash Table Search is a strong choice for exact registration-number lookup. However, the system should use a reliable, maintained index rather than depend on an unindexed scan of all student records.

C. Adaptation or Redesign

C.1 Proposed Adapted Design

The design will use a hash-based index for exact registration-number searches, with a daily update process to incorporate new student records. The proposed workflow is:

1. Store student information in the main database.
2. Create an index that maps each unique registration number to its record or database identifier.
3. Receive a search request and validate the registration number.
4. Use the index to locate the corresponding record.
5. Return the student information if a matching record exists.
6. Return a suitable message if the registration number is not found.
7. Add newly registered students and update the index during the daily update process.
8. Monitor lookup latency, memory usage, collisions, and failed searches.

C.2 Why the Adapted Design Is Better

The adapted design avoids scanning all student records for every search. It supports frequent exact-key searches and separates the cost of preparing the index from the cost of using it. Since new records are added only once per day, the index can be updated in a scheduled process while the system ensures that new records become searchable after the update completes. A database-backed index is preferable in a production environment because it can support persistence, concurrent access, and reliable updates. The exact index implementation should be chosen according to the database technology and operational requirements.

C.3 Handling the Daily Update

The daily update process should:

- Validate new registration numbers and ensure that they are unique.
- Add valid student records to the main database.
- Update the hash-based index or database index.
- Verify that the new records can be searched.
- Record errors and recover safely if an update fails.
- Ensure that searches use a consistent version of the index during the update. These steps help maintain accurate results while allowing the system to handle regular searches.

C.4 Performance and Resource Considerations

Hash Table Search has expected O(1) lookup time, but it requires additional memory and proper collision handling. The system should monitor memory consumption as the number of records grows. The average workload is approximately 13.89 searches per second. This rate alone does not guarantee that the system will meet response-time requirements, because actual performance also depends on database access, network delay, concurrent requests, and hardware capacity. Therefore, the adapted design should be tested under realistic conditions before deployment.

D. Evidence-Based Defence

D.1 Defence Using Time Complexity

The adapted design is supported by the theoretical complexity of the algorithms.

Algorithm	**Worst-Case Search Complexity**	**Suitability for Updated Workload**
Linear Search	O(n)	Poor as the primary lookup method
Recursive Binary Search	O(log n)	Suitable when data is sorted
Hash Table Search	O(n) worst case; expected O(1)	Strong option for exact-key lookups

For one million records, Linear Search may examine all one million entries in the worst case. Binary Search requires approximately 20 halving steps. Hash Table Search is expected to locate the corresponding entry with a small amount of work under suitable hashing conditions. These theoretical results support using an index-based approach for the updated workload.

D.2 Defence Using Workload Requirements

The system must handle 50,000 searches per hour and one million records. Because the average request rate is approximately 14 searches per second, the main searching structure should provide fast repeated lookups. The daily update schedule also makes it reasonable to maintain an index without rebuilding it for every search. The design should update the index when new records are added and then serve searches using the maintained index.

D.3 Defence Using Experimental Evidence

The implementation should be tested with one million records and a workload representing 50,000 searches. Testing should measure:

- Average and maximum search latency.

- Total time to complete the workload.
- Number of successful and unsuccessful searches.
- Hash-table or index construction and update time.
- Memory consumption.
- Performance under concurrent requests. The measured results should be compared with the theoretical expectations. No experimental performance values should be claimed until the tests have actually been executed.

D.4 Risks and Mitigation

Risk 1: Hash collisions. Poor hash distribution may increase search time. Use a reliable hash-table implementation and monitor its performance.

Risk 2: Memory usage. The index requires additional memory. Measure memory requirements using the full dataset and ensure that sufficient capacity is available.

Risk 3: Inconsistent daily updates. An incomplete update may make new records unavailable. Use validation, error handling, and a controlled update process.

Risk 4: Increased peak traffic. The average rate may hide short periods of heavy demand. Test the system under realistic peak workloads.

Risk 5: Incorrect or duplicate registration numbers. Validate new records and enforce uniqueness before updating the index.

D.5 Final Defence

Hash Table Search remains the recommended strategy for exact student registration-number searches because it provides expected constant-time lookup and suits a workload with many searches and relatively infrequent updates. Recursive Binary Search remains useful where sorted retrieval is required. Linear Search can still be used for small collections or simple verification, but it should not be the main lookup method for one million records. The final design should be accepted only after testing confirms that it handles the expected workload, meets the required response times, and operates within available memory and processing resources.

12.2 Final Conclusion

The updated scenario changes the system from a general searching problem into a high-frequency lookup problem involving one million student records and 50,000 searches per hour. The addition of new records only once per day makes scheduled index updates practical. The analysis supports adapting the original design to use a maintained hash-based index for exact registration-number searches. This design reduces the need for repeated full-database scans and provides expected constant-time lookup under suitable conditions. The system must still account for memory usage, collision handling, daily updates, concurrency,

and real-world response times. The final decision should be defended using both theoretical complexity and actual experimental measurements.